

JAVA PROGRAMMING

Chapter 6: Conditionals in Java

By Akanshu Jamwal

Table of Contents

1. Introduction to Conditionals
2. if Statement
3. if-else Statement
4. if-else-if Ladder
5. Nested if-else
6. Ternary Operator
7. Switch Statement
8. break Keyword in Switch
9. Switch Fall-Through Behavior
10. When to Use Which Conditional
11. Common Mistakes
12. Practice Challenge

1. Introduction to Conditionals

In programming, **decisions drive logic**. Conditional statements allow your program to make decisions and execute different code based on different conditions.

Real-world Examples:

- If a student completes all modules → unlock advanced content
- If score ≥ 90 → assign Grade A
- If user is premium → enable features
- If age ≥ 18 → allow access

Types of Conditionals in Java:

- if statement
- if-else statement
- if-else-if ladder
- Nested if-else
- Ternary operator
- Switch statement

2. if Statement

The if statement executes a block of code **only if a condition is true**.

Syntax:

```
if (condition) {  
    // code executes if condition is true  
}
```

Example:

```
int dailyPractice = 12;

if (dailyPractice >= 10) {
    System.out.println("Good consistency!");
}
// Since 12 >= 10 is true, output: Good consistency!
```

3. if-else Statement

The if-else statement executes one block if condition is true, and another block if false.

Syntax:

```
if (condition) {
    // code if true
} else {
    // code if false
}
```

Example:

```
int score = 42;

if (score >= 50) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
// Since 42 >= 50 is false, output: Fail
```

4. if-else-if Ladder

Used when you have **multiple conditions** to check. The first true condition executes, others are skipped.

Syntax:

```
if (condition1) {  
    // code  
} else if (condition2) {  
    // code  
} else if (condition3) {  
    // code  
} else {  
    // code  
}
```

Example: Grade Assignment

```
int accuracy = 78;  
  
if (accuracy >= 90) {  
    System.out.println("Excellent");  
} else if (accuracy >= 75) {  
    System.out.println("Good");  
} else if (accuracy >= 60) {  
    System.out.println("Average");  
} else {  
    System.out.println("Needs Improvement");  
}  
// Output: Good (first true condition)
```

5. Nested if-else

An if-else statement **inside another if-else statement**. Used when conditions depend on each other.

Example: Access Control

```
boolean hasSubscription = true;
int solvedProblems = 220;

if (hasSubscription) {
    if (solvedProblems >= 200) {
        System.out.println("Unlock Advanced Sheet");
    } else {
        System.out.println("Practice More");
    }
} else {
    System.out.println("Upgrade to Premium");
}
// Output: Unlock Advanced Sheet
```

6. Ternary Operator

A shortcut for simple if-else statements. It's a **one-liner** that makes code concise.

Syntax:

```
condition ? valueIfTrue : valueIfFalse
```

Example:

```
int streakDays = 35;

String status = (streakDays >= 30) ? "Consistent" : "Irregular";

System.out.println(status); // Output: Consistent
```

7. Switch Statement

Used when a **single variable** has multiple possible values. Much cleaner than long if-else ladders.

Syntax:

```
switch (expression) {  
    case value1:  
        // code  
        break;  
    case value2:  
        // code  
        break;  
    default:  
        // code  
}
```

Rules of Switch:

- Expression must be: int, char, String, or enum
- case values must be constants
- break stops execution
- default is optional but recommended

Example: Day of Week

```
int dayNumber = 3;  
  
switch (dayNumber) {  
    case 1:  
        System.out.println("Monday");  
        break;  
    case 2:  
        System.out.println("Tuesday");  
        break;  
    case 3:  
        System.out.println("Wednesday");  
        break;  
    default:  
        System.out.println("Invalid");  
}  
// Output: Wednesday
```

8. break Keyword in Switch

The `break` keyword **stops execution of the switch**. Without it, code continues to the next case (fall-through).

Important:

Forgetting `break` is one of the most common mistakes in switch statements!

9. Switch Fall-Through Behavior

When `break` is missing, execution **continues** to the next case. This is called fall-through.

Example WITHOUT `break`:

```
int level = 2;

switch (level) {
    case 1:
        System.out.println("Beginner");
    case 2:
        System.out.println("Intermediate");
    case 3:
        System.out.println("Advanced");
}

Output:
Intermediate
Advanced
```

Notice: Both **Intermediate** and **Advanced** printed because there's no `break` after case 2!

10. When to Use Which Conditional

Conditional	Best For
if	Single condition (true/false)
if-else	Two alternative paths
if-else-if	Multiple conditions on same variable
Nested if-else	Dependent conditions
Ternary	Quick assignment (one-liner)
Switch	Many cases on single variable

11. Common Mistakes

Mistakes often made in interviews:

- Missing break in switch → causes fall-through
- Wrong condition ordering in if-else-if
- Using == for String comparison (use .equals())
- Incorrect nesting of conditions
- Confusing ternary operator syntax
- Not handling all cases in switch

■■ String Comparison:

WRONG: `if (name == "Aman")` // compares references

CORRECT: `if (name.equals("Aman"))` // compares values

12. Practice Challenge

Build a Program:

Input: number of problems solved

Output:

- $\geq 300 \rightarrow$ "Advanced Level"
- $\geq 150 \rightarrow$ "Intermediate Level"
- $< 150 \rightarrow$ "Beginner Level"

Challenge:

1. Use if-else-if ladder
2. Use switch statement (convert the range to categories)
3. Use ternary operator for one condition

Compare which approach is cleaner for this problem!

What's Next?

Now that you understand conditionals, you're ready for loops and iterative logic:

- for loop - iterate with counter
- while loop - iterate while condition is true
- do-while loop - execute at least once
- Pattern problems and problem solving
- Logical thinking for DSA

■ Key Insight:

Conditionals + Loops = Foundation of programming. Master these concepts early to solve complex problems effectively!